

yolov8-realwaste.ipynb

File

waste_classification.ipynb...

File

build_cnn.ipynb

File

take some time to remember this project and dataset. If you are unfamiliar with this project, which you may be, please review the project proposal here

Waste Classification Final Project Proposal

About

In image classification dataset to classify waste type.

Outcome

Compare and Analyze several different CNN models, tune parameters, and utilize data augmentation.

Show more ▾

I'm opening the notebook and extracting it cell by cell so I can reproduce it cleanly here in the chat, including the markdown cells and code cells in order.

I found one spot where the terminal view truncated part of the notebook, so I'm extracting the full raw cell contents directly to avoid missing anything.

Thought for 37s >

Absolutely — below is the notebook written out **cell by cell**, including every markdown cell and code cell in order, so it can be printed from this chat and referenced directly.

I'm reproducing the content as notebook cells, not as a shortened summary.

Cell 1 — Markdown

RealWaste CNN Classification Notebook

This notebook is designed for your **waste classification final project** and is meant to be:

1. a **strong training notebook** focused on **high classification accuracy**, and
2. a **learning notebook** that explains **why** each step exists so you can use it in

Project goals

We want to:

- classify waste images from the **RealWaste** dataset,
- compare several CNN backbones,
- improve accuracy with transfer learning, augmentation, and tuning,
- keep careful notes about what helped and what did not,
- prepare the project for a later upgrade to **YOLO/object detection**.

Why this notebook is different from the earlier notebooks

The sample notebooks were useful starting points, but for this project we want a more

- **stratified train/validation/test split** instead of a plain random split,
- **clear experiment tracking** so you can report results,
- **modern pretrained CNN backbones** instead of training from scratch,
- **stronger data augmentation** because the dataset is relatively small,
- **class imbalance handling** because some classes have many more images than others,
- **confusion matrix and per-class metrics** so we can see exactly where mistakes happen.

This notebook is written step by step so you can learn as you go.

Cell 2 — Markdown

Recommended workflow for the project

Phase 1: Strong image classifier first

Start with image classification using one object per image. This is the fastest way to

Phase 2: Improve accuracy carefully

We will test changes one at a time:

- augmentation,
- image size,

- class weighting,
- better pretrained backbones,
- learning-rate scheduling,
- freezing vs. unfreezing layers,
- optional MixUp / CutMix.

Phase 3: Transition to YOLO later

YOLO is for **object detection**, not just classification. That means you will need:

- bounding boxes,
- possibly multiple objects per image,
- a different dataset format.

So the right order is:

classification first -> understand class confusion -> then build object detection

Cell 3 — Markdown

Environment setup

Create a virtual environment first, then install the packages you need.

Example:

```
```bash
python -m venv .venv
source .venv/bin/activate # macOS/Linux
.venv\Scripts\activate # Windows

pip install --upgrade pip
pip install torch torchvision torchaudio
pip install scikit-learn pandas matplotlib seaborn pillow tqdm
```

Optional later:

```
pip install jupyterlab
pip install ultralytics # for YOLO later
```

If you are using Apple Silicon (M1/M2/M3), this notebook can also use **MPS**.

## Cell 4 — Code



```
```python
# If you need to install packages inside the notebook, uncomment these lines:
# %pip install torch torchvision torchaudio scikit-learn pandas matplotlib sea
```

Cell 5 — Markdown

```
## 1. Imports and configuration
```

We keep most project settings in one place so it is easy to change things later and e

Cell 6 — Code

```
import os
import copy
import json
import math
import random
import time
from dataclasses import dataclass, asdict
from pathlib import Path
from collections import Counter, defaultdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from PIL import Image

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler

import torchvision
from torchvision import transforms
from torchvision.datasets import ImageFolder
from torchvision.models import (
    resnet50, ResNet50_Weights,
```

```
mobilenet_v3_large, MobileNet_V3_Large_Weights,
efficientnet_b0, EfficientNet_B0_Weights,
efficientnet_b2, EfficientNet_B2_Weights,
)

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
)

from tqdm.auto import tqdm

plt.rcParams['figure.figsize'] = (8, 5)
plt.rcParams['axes.grid'] = True
```

Cell 7 — Markdown

2. Reproducibility and device

Why this matters:

- Reproducibility makes experiments fair.
- Device detection lets the notebook work on CPU, CUDA GPU, or Apple MPS.

Cell 8 — Code

```
SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = False
    torch.backends.cudnn.benchmark = True

set_seed(SEED)

if torch.cuda.is_available():
    DEVICE = torch.device("cuda")
```

```
elif torch.backends.mps.is_available():  
    DEVICE = torch.device("mps")  
else:  
    DEVICE = torch.device("cpu")  
  
print(f"Using device: {DEVICE}")
```

Cell 9 — Markdown

3. Set your dataset path

This notebook assumes the dataset is arranged like this:

```
```text  
RealWaste/
 Cardboard/
 FoodOrganics/
 Glass/
 Metal/
 MiscellaneousTrash/
 Paper/
 Plastic/
 TextileTrash/
 Vegetation/
```

If your folder names are slightly different, that is okay. The notebook will read class names from the directory structure.



```
Cell 10 – Code
```python  
# CHANGE THIS to wherever your RealWaste dataset is stored.  
DATA_DIR = Path("./RealWaste")  
  
assert DATA_DIR.exists(), f"Dataset folder not found: {DATA_DIR.resolve()}"  
print("Dataset directory:", DATA_DIR.resolve())
```

Cell 11 — Markdown

4. Build a dataframe of file paths and labels

Why do this instead of relying only on ``ImageFolder``?

Because it gives us more control:

- we can make a **stratified split**,
- inspect class imbalance,
- save file-based train/val/test splits for reproducibility,
- later reuse the same split for future models.

Cell 12 — Code

```
IMG_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

records = []
for class_dir in sorted([p for p in DATA_DIR.iterdir() if p.is_dir()]):
    for img_path in sorted(class_dir.rglob("*")):
        if img_path.suffix.lower() in IMG_EXTS:
            records.append({"filepath": str(img_path), "label": class_dir.name})

_df = pd.DataFrame(records)
assert len(_df) > 0, "No images found. Check DATA_DIR and folder structure."

print(f"Total images found: {len(_df):,}")
display(_df.head())
```

Cell 13 — Markdown

5. Inspect class balance

This is important because your dataset is **not perfectly balanced**.

That matters because a model can look "good" overall while still doing poorly on sma

Cell 14 — Code

```
class_counts = _df['label'].value_counts().sort_index()
display(class_counts.to_frame(name='count'))

plt.figure(figsize=(10, 5))
```

```
class_counts.plot(kind='bar')
plt.title('Class Distribution')
plt.ylabel('Number of Images')
plt.xlabel('Class')
plt.xticks(rotation=45, ha='right')
plt.show()
```

Cell 15 — Markdown

Notes for your report

You can say something like:

- The dataset is moderately imbalanced.
- Plastic and Metal have many more samples than Textile Trash or Food Organics.
- This motivates using **class weighting** and/or **weighted sampling** during training.

Cell 16 — Markdown

6. Stratified train / validation / test split

This is one of the most important improvements over a plain random split.

A stratified split tries to preserve class proportions across each split. That gives a **fairer validation set** and **fairer test set**.

Cell 17 — Code

```
train_df, temp_df = train_test_split(
    _df,
    test_size=0.20,
    stratify=_df['label'],
    random_state=SEED,
)

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
```

```

        stratify=temp_df['label'],
        random_state=SEED,
    )

    print(f"Train size: {len(train_df)}")
    print(f"Val size: {len(val_df)}")
    print(f"Test size: {len(test_df)}")

    split_summary = pd.DataFrame({
        'train': train_df['label'].value_counts().sort_index(),
        'val': val_df['label'].value_counts().sort_index(),
        'test': test_df['label'].value_counts().sort_index(),
    }).fillna(0).astype(int)
    display(split_summary)

```

Cell 18 — Code

```

# Save the split so future experiments use exactly the same images.
SPLIT_DIR = Path("./splits")
SPLIT_DIR.mkdir(exist_ok=True)

train_df.to_csv(SPLIT_DIR / "train_split.csv", index=False)
val_df.to_csv(SPLIT_DIR / "val_split.csv", index=False)
test_df.to_csv(SPLIT_DIR / "test_split.csv", index=False)

print("Saved split files to:", SPLIT_DIR.resolve())

```

Cell 19 — Markdown

7. Create label mappings

We convert class names into integer indices because models train on numbers, not strings.

Cell 20 — Code

```

class_names = sorted(df['label'].unique().tolist())
class_to_idx = {name: i for i, name in enumerate(class_names)}
idx_to_class = {i: name for name, i in class_to_idx.items()}

```

```
print(class_to_idx)
NUM_CLASSES = len(class_names)
print("Number of classes:", NUM_CLASSES)
```

Cell 21 — Markdown

8. Define image transforms

Why data augmentation matters here

RealWaste is not a huge dataset, so augmentation helps the model see more variety. This can reduce overfitting and improve generalization.

Good augmentations for this task

Useful:

- horizontal flip,
- small rotations,
- random resized crop,
- slight color jitter,
- small perspective / affine changes.

Usually avoid extreme transforms that make waste objects unrealistic.

Important note

Vertical flips are not always ideal for all real-world objects. They can still help s

Cell 22 — Code

```
# Change image size here if you want to experiment.
# 224 is standard and efficient. 260 or 288 can sometimes help slightly.
IMAGE_SIZE = 224

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.75, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.10, hue=0.02),
    transforms.RandomPerspective(distortion_scale=0.15, p=0.2),
    transforms.ToTensor(),
```

```
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

val_test_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

Cell 23 — Markdown

```
## 9. Custom dataset class using the saved dataframes
```

Cell 24 — Code

```
class WasteDataset(Dataset):
    def __init__(self, df, class_to_idx, transform=None):
        self.df = df.reset_index(drop=True).copy()
        self.class_to_idx = class_to_idx
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        image = Image.open(row['filepath']).convert('RGB')
        label = self.class_to_idx[row['label']]

        if self.transform is not None:
            image = self.transform(image)

        return image, label
```

Cell 25 — Markdown

```
## 10. Build datasets and dataloaders
```

We will use a **WeightedRandomSampler** during training.

Why this helps

If one class has far fewer samples, the sampler makes it more likely those images st:
This is one way to reduce the effect of imbalance.

Cell 26 — Code

```
train_dataset = WasteDataset(train_df, class_to_idx, transform=train_transform)
val_dataset = WasteDataset(val_df, class_to_idx, transform=val_test_transform)
test_dataset = WasteDataset(test_df, class_to_idx, transform=val_test_transform)

train_class_counts = train_df['label'].value_counts().sort_index()
train_sample_weights = train_df['label'].map(lambda x: 1.0 / train_class_counts[x]).v
train_sample_weights = torch.DoubleTensor(train_sample_weights)

weighted_sampler = WeightedRandomSampler(
    weights=train_sample_weights,
    num_samples=len(train_sample_weights),
    replacement=True,
)

BATCH_SIZE = 32
NUM_WORKERS = 2
PIN_MEMORY = DEVICE.type == 'cuda'

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    sampler=weighted_sampler,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY,
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY,
)

test_loader = DataLoader(
    test_dataset,
```

```
batch_size=BATCH_SIZE,  
shuffle=False,  
num_workers=NUM_WORKERS,  
pin_memory=PIN_MEMORY,  
)  
  
print("Data loaders ready.")
```

Cell 27 — Markdown

11. Visualize a few augmented training images

This is important because it lets you check whether the augmentations are helpful or

Cell 28 — Code

```
def denormalize(img_tensor, mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)):  
    mean = torch.tensor(mean).view(3, 1, 1)  
    std = torch.tensor(std).view(3, 1, 1)  
    return img_tensor.cpu() * std + mean  
  
images, labels = next(iter(train_loader))  
  
fig, axes = plt.subplots(2, 4, figsize=(14, 7))  
axes = axes.flatten()  
for ax, image, label in zip(axes, images[:8], labels[:8]):  
    img = denormalize(image).clamp(0, 1)  
    ax.imshow(img.permute(1, 2, 0))  
    ax.set_title(idx_to_class[int(label)])  
    ax.axis('off')  
plt.tight_layout()  
plt.show()
```

Cell 29 — Markdown

12. Class weights for the loss function

We are going to combine two ideas:

- **Weighted sampling** to show minority classes more often,
- **Class-weighted loss** so mistakes on minority classes matter more.

This does not always help, but it is a good experiment for an imbalanced dataset.

Cell 30 — Code

```
train_counts_vector = np.array([train_class_counts[name] for name in class_names], dtype=np.float32)
class_weights = train_counts_vector.sum() / (len(train_counts_vector) * train_counts_vector)
class_weights = torch.tensor(class_weights, dtype=torch.float32, device=DEVICE)

print("Class weights:")
for name, weight in zip(class_names, class_weights.cpu().numpy()):
    print(f"{name:>22}: {weight:.3f}")
```

Cell 31 — Markdown

13. Model factory

We will compare several pretrained CNN backbones:

- **ResNet50**
- **MobileNetV3-Large**
- **EfficientNet-B0**
- **EfficientNet-B2**

Why pretrained models?

They already know useful visual patterns from ImageNet, so they usually perform much

Suggested strategy

1. Train the classifier head first with the backbone frozen.
2. Then unfreeze the backbone and fine-tune the full network.

That is usually a strong approach for accuracy on smaller datasets.

Cell 32 — Code


```

def build_model(model_name: str, num_classes: int, freeze_backbone: bool = True):
    model_name = model_name.lower()

    if model_name == 'resnet50':
        weights = ResNet50_Weights.DEFAULT
        model = resnet50(weights=weights)
        in_features = model.fc.in_features
        model.fc = nn.Sequential(
            nn.Dropout(0.30),
            nn.Linear(in_features, num_classes)
        )

    elif model_name == 'mobilenet_v3_large':
        weights = MobileNet_V3_Large_Weights.DEFAULT
        model = mobilenet_v3_large(weights=weights)
        in_features = model.classifier[-1].in_features
        model.classifier[-1] = nn.Linear(in_features, num_classes)

    elif model_name == 'efficientnet_b0':
        weights = EfficientNet_B0_Weights.DEFAULT
        model = efficientnet_b0(weights=weights)
        in_features = model.classifier[-1].in_features
        model.classifier = nn.Sequential(
            nn.Dropout(0.30),
            nn.Linear(in_features, num_classes)
        )

    elif model_name == 'efficientnet_b2':
        weights = EfficientNet_B2_Weights.DEFAULT
        model = efficientnet_b2(weights=weights)
        in_features = model.classifier[-1].in_features
        model.classifier = nn.Sequential(
            nn.Dropout(0.35),
            nn.Linear(in_features, num_classes)
        )

    else:
        raise ValueError(f"Unknown model name: {model_name}")

    if freeze_backbone:
        for param in model.parameters():
            param.requires_grad = False

        # Unfreeze only the classifier head
        if model_name == 'resnet50':
            for param in model.fc.parameters():

```

```

        param.requires_grad = True
    else:
        for param in model.classifier.parameters():
            param.requires_grad = True

    return model

```

Cell 33 — Markdown

14. Training utilities

This section contains the reusable training loop.

Accuracy-focused decisions used here

- **CrossEntropyLoss** with label smoothing: often helps generalization.
- **AdamW**: good optimizer for transfer learning.
- **ReduceLROnPlateau** scheduler: lowers learning rate when validation stops improving.
- **Early stopping**: prevents wasting epochs after the model stops improving.
- **Macro F1 score**: useful because the classes are imbalanced.

Cell 34 — Code

```

@dataclass
class TrainConfig:
    model_name: str = 'efficientnet_b2'
    epochs_head: int = 5
    epochs_finetune: int = 12
    lr_head: float = 1e-3
    lr_finetune: float = 2e-4
    weight_decay: float = 1e-4
    label_smoothing: float = 0.05
    patience: int = 5
    use_class_weights: bool = True

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    all_preds, all_targets = [], []

    for images, targets in tqdm(loader, leave=False):

```

```

images, targets = images.to(device), targets.to(device)

optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, targets)
loss.backward()
optimizer.step()

running_loss += loss.item() * images.size(0)
preds = outputs.argmax(dim=1)
all_preds.extend(preds.detach().cpu().numpy())
all_targets.extend(targets.detach().cpu().numpy())

epoch_loss = running_loss / len(loader.dataset)
epoch_acc = accuracy_score(all_targets, all_preds)
epoch_f1 = f1_score(all_targets, all_preds, average='macro')
return epoch_loss, epoch_acc, epoch_f1

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    all_preds, all_targets = [], []

    for images, targets in tqdm(loader, leave=False):
        images, targets = images.to(device), targets.to(device)
        outputs = model(images)
        loss = criterion(outputs, targets)

        running_loss += loss.item() * images.size(0)
        preds = outputs.argmax(dim=1)
        all_preds.extend(preds.detach().cpu().numpy())
        all_targets.extend(targets.detach().cpu().numpy())

    epoch_loss = running_loss / len(loader.dataset)
    epoch_acc = accuracy_score(all_targets, all_preds)
    epoch_f1 = f1_score(all_targets, all_preds, average='macro')
    return epoch_loss, epoch_acc, epoch_f1, np.array(all_targets), np.array(all_preds)

```

Cell 35 — Code

```

def run_training(config: TrainConfig, train_loader, val_loader, device=DEVICE):
    model = build_model(config.model_name, NUM_CLASSES, freeze_backbone=True).to(device)

```

```

criterion = nn.CrossEntropyLoss(
    weight=class_weights if config.use_class_weights else None,
    label_smoothing=config.label_smoothing,
)

history = []
best_state = None
best_val_f1 = -np.inf
no_improve_count = 0

# -----
# Phase 1: train classifier head only
# -----
optimizer = optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=config.lr_head,
    weight_decay=config.weight_decay,
)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='max', factor=0.5, patience=2
)

total_epochs = config.epochs_head + config.epochs_finetune

for epoch in range(1, config.epochs_head + 1):
    train_loss, train_acc, train_f1 = train_one_epoch(model, train_loader, criterion)
    val_loss, val_acc, val_f1, _, _ = evaluate(model, val_loader, criterion, device)
    scheduler.step(val_f1)

    row = {
        'epoch': epoch,
        'phase': 'head',
        'train_loss': train_loss,
        'train_acc': train_acc,
        'train_f1': train_f1,
        'val_loss': val_loss,
        'val_acc': val_acc,
        'val_f1': val_f1,
        'lr': optimizer.param_groups[0]['lr'],
    }
    history.append(row)
    print(row)

    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        best_state = copy.deepcopy(model.state_dict())
        no_improve_count = 0
    else:

```

```

        no_improve_count += 1

# -----
# Phase 2: fine-tune all layers
# -----
for param in model.parameters():
    param.requires_grad = True

optimizer = optim.AdamW(
    model.parameters(),
    lr=config.lr_finetune,
    weight_decay=config.weight_decay,
)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='max', factor=0.5, patience=2
)

for epoch in range(config.epochs_head + 1, total_epochs + 1):
    train_loss, train_acc, train_f1 = train_one_epoch(model, train_loader, criterion)
    val_loss, val_acc, val_f1, _, _ = evaluate(model, val_loader, criterion, device)
    scheduler.step(val_f1)

    row = {
        'epoch': epoch,
        'phase': 'finetune',
        'train_loss': train_loss,
        'train_acc': train_acc,
        'train_f1': train_f1,
        'val_loss': val_loss,
        'val_acc': val_acc,
        'val_f1': val_f1,
        'lr': optimizer.param_groups[0]['lr'],
    }
    history.append(row)
    print(row)

    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        best_state = copy.deepcopy(model.state_dict())
        no_improve_count = 0
    else:
        no_improve_count += 1

    if no_improve_count >= config.patience:
        print(f"Early stopping triggered after epoch {epoch}.")
        break

model.load_state_dict(best_state)

```

```
history_df = pd.DataFrame(history)
return model, history_df
```

Cell 36 — Markdown

15. Quick model comparison

This cell lets you compare several backbones using the same pipeline.

Recommendation

Start with:

- `efficientnet\_b0`
- `resnet50`
- `mobilenet\_v3\_large`

If you have enough compute, add:

- `efficientnet\_b2`

In many small-to-medium image classification tasks, **EfficientNet** often performs \ **MobileNet** is lighter and faster.

ResNet50 is a very strong classic baseline.

Cell 37 — Code

```
MODEL_CANDIDATES = [
    'mobilenet_v3_large',
    'resnet50',
    'efficientnet_b0',
    'efficientnet_b2',
]
```

```
BASE_CONFIG = TrainConfig(
    epochs_head=4,
    epochs_finetune=8,
    lr_head=1e-3,
    lr_finetune=2e-4,
    weight_decay=1e-4,
    label_smoothing=0.05,
    patience=4,
```

```

        use_class_weights=True,
    )

```

Cell 38 — Code

```

# Run this cell to compare models.
# It may take a while depending on your hardware.

comparison_rows = []
all_histories = {}
trained_models = {}

for model_name in MODEL_CANDIDATES:
    print("
" + "=" * 80)
    print(f"Training {model_name}")
    print("=" * 80)

    cfg = copy.deepcopy(BASE_CONFIG)
    cfg.model_name = model_name

    model, history_df = run_training(cfg, train_loader, val_loader, device=DEVICE)
    trained_models[model_name] = model
    all_histories[model_name] = history_df

    best_row = history_df.sort_values('val_f1', ascending=False).iloc[0]
    comparison_rows.append({
        'model_name': model_name,
        'best_val_acc': best_row['val_acc'],
        'best_val_f1': best_row['val_f1'],
        'best_epoch': int(best_row['epoch']),
    })

comparison_df = pd.DataFrame(comparison_rows).sort_values('best_val_f1', ascending=False)
display(comparison_df)

```

Cell 39 — Markdown

If you want a faster first run

You can comment out the comparison loop above and train only one model first, for example:

- `efficientnet\_b0` for a faster strong baseline,
- or `efficientnet\_b2` if accuracy is your top priority and compute is available.

Cell 40 — Code

```
# Example: train only one chosen model.
# Uncomment and run if you want a single focused training run.

# chosen_config = TrainConfig(
#     model_name='efficientnet_b2',
#     epochs_head=5,
#     epochs_finetune=12,
#     lr_head=1e-3,
#     lr_finetune=2e-4,
#     weight_decay=1e-4,
#     label_smoothing=0.05,
#     patience=5,
#     use_class_weights=True,
# )
# best_model, best_history = run_training(chosen_config, train_loader, val_loader, de
```

Cell 41 — Markdown

16. Pick the best model

If you used the comparison section, this cell automatically chooses the model with the

Cell 42 — Code

```
if 'comparison_df' in globals() and len(comparison_df) > 0:
    best_model_name = comparison_df.iloc[0]['model_name']
    best_model = trained_models[best_model_name]
    best_history = all_histories[best_model_name]
    print(f"Selected best model: {best_model_name}")
```



```
else:  
    print("Run the comparison section above or assign best_model manually.")
```

Cell 43 — Markdown

17. Plot training curves

These plots help answer report questions like:

- Did the model overfit?
- Did the scheduler help?
- Did fine-tuning help after the frozen-head stage?

Cell 44 — Code

```
if 'best_history' in globals():  
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))  
  
    axes[0].plot(best_history['epoch'], best_history['train_loss'], label='Train Loss')  
    axes[0].plot(best_history['epoch'], best_history['val_loss'], label='Val Loss')  
    axes[0].set_title('Loss Curves')  
    axes[0].set_xlabel('Epoch')  
    axes[0].set_ylabel('Loss')  
    axes[0].legend()  
  
    axes[1].plot(best_history['epoch'], best_history['train_acc'], label='Train Acc')  
    axes[1].plot(best_history['epoch'], best_history['val_acc'], label='Val Acc')  
    axes[1].plot(best_history['epoch'], best_history['val_f1'], label='Val Macro F1')  
    axes[1].set_title('Accuracy / F1 Curves')  
    axes[1].set_xlabel('Epoch')  
    axes[1].set_ylabel('Score')  
    axes[1].legend()  
  
    plt.tight_layout()  
    plt.show()
```

Cell 45 — Markdown

18. Final evaluation on the test set

Do this **only after** choosing your final model using the validation set.
That keeps the test set honest.

Cell 46 — Code

```
criterion_for_eval = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.0)

test_loss, test_acc, test_f1, y_true, y_pred = evaluate(
    best_model, test_loader, criterion_for_eval, DEVICE
)

print(f"Test Loss:      {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test Macro F1:   {test_f1:.4f}")
```

Cell 47 — Code

```
report = classification_report(
    y_true,
    y_pred,
    target_names=class_names,
    digits=4,
    output_dict=True,
)
report_df = pd.DataFrame(report).transpose()
display(report_df)
```

Cell 48 — Code

```
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.xticks(rotation=45, ha='right')
```

```
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

Cell 49 — Markdown

How to interpret the confusion matrix

This is one of the most important analysis tools for your report.

Look for questions like:

- Does **Paper** get confused with **Cardboard**?
- Does **MiscellaneousTrash** absorb difficult examples from other classes?
- Are organic-looking classes such as **FoodOrganics** and **Vegetation** getting mixed up?

This gives you ideas for future improvements:

- more examples,
- cleaner labels,
- stronger augmentation,
- larger image size,
- better class definitions,
- or eventually object detection if clutter/background is causing errors.

Cell 50 — Markdown

19. Show sample predictions

Seeing correct and incorrect examples is extremely useful for learning *why* the model is wrong.

Cell 51 — Code

```
@torch.no_grad()
def show_predictions(model, loader, idx_to_class, device=DEVICE, max_images=12):
    model.eval()
    images, labels = next(iter(loader))
```

```

images = images.to(device)
outputs = model(images)
preds = outputs.argmax(dim=1).cpu()

fig, axes = plt.subplots(math.ceil(max_images / 4), 4, figsize=(14, 3 * math.ceil(max_images / 4)))
axes = np.array(axes).reshape(-1)

for i in range(max_images):
    img = denormalize(images[i].cpu()).clamp(0, 1)
    axes[i].imshow(img.permute(1, 2, 0))
    true_name = idx_to_class[int(labels[i])]
    pred_name = idx_to_class[int(preds[i])]
    color = 'green' if true_name == pred_name else 'red'
    axes[i].set_title(f"Pred: {pred_name}
True: {true_name}", color=color)
    axes[i].axis('off')

for ax in axes[max_images:]:
    ax.axis('off')

plt.tight_layout()
plt.show()

show_predictions(best_model, test_loader, idx_to_class)

```

Cell 52 — Markdown

20. Save the best model and experiment results

This is helpful for your final report, final presentation, and later deployment work.

Cell 53 — Code

```

OUTPUT_DIR = Path('./outputs')
OUTPUT_DIR.mkdir(exist_ok=True)

best_model_path = OUTPUT_DIR / 'best_realwaste_model.pt'
torch.save(best_model.state_dict(), best_model_path)
print('Saved model to:', best_model_path.resolve())

```

```

if 'best_history' in globals():
    best_history.to_csv(OUTPUT_DIR / 'best_history.csv', index=False)

if 'comparison_df' in globals():
    comparison_df.to_csv(OUTPUT_DIR / 'model_comparison.csv', index=False)

report_df.to_csv(OUTPUT_DIR / 'classification_report.csv')

summary = {
    'test_loss': float(test_loss),
    'test_accuracy': float(test_acc),
    'test_macro_f1': float(test_f1),
    'class_names': class_names,
    'image_size': IMAGE_SIZE,
    'batch_size': BATCH_SIZE,
    'device': str(DEVICE),
}

with open(OUTPUT_DIR / 'run_summary.json', 'w') as f:
    json.dump(summary, f, indent=2)

print('Saved outputs to:', OUTPUT_DIR.resolve())

```

Cell 54 — Markdown

21. Experiment notes template for your report

Use this section to record what changed and whether it helped.

A strong final report usually includes an ablation-style table like this:

Experiment	Change	Why we tried it	Val Accuracy	Val Macro F1	Result
Baseline	EfficientNet-B0, resize 224, no weighted loss	strong transfer baseline			
Exp 2	Added stronger augmentation	reduce overfitting	...	...	helped / did not
Exp 3	Added weighted sampler + class weights	address imbalance	...	...	helped / did not
Exp 4	Switched to EfficientNet-B2	larger backbone	...	...	helped / did not
Exp 5	Increased image size	capture more detail	...	...	helped / did not

What usually helps on this dataset

Most likely helpful:

- transfer learning,
- careful augmentation,
- stronger backbone,
- class imbalance handling,

- fine-tuning after frozen-head training,
- confusion-matrix-driven error analysis.

What might not help

Sometimes these hurt performance:

- augmentation that is too aggressive,
- too many epochs with no early stopping,
- training from scratch,
- too large a learning rate,
- vertical flips if they make images unrealistic,
- evaluating many settings without keeping the split fixed.

Cell 55 — Markdown

22. Accuracy improvement ideas to try next

Try these one at a time so you know what caused the improvement.

Strong next experiments

1. **Increase image size** from 224 to 260 or 288
2. **Train longer** with patience increased slightly
3. **Try EfficientNet-B2 vs B0 carefully**
4. **Test with and without class weights**
5. **Test with and without weighted sampler**
6. **Tune augmentation strength**
7. **Try label smoothing = 0.0, 0.05, 0.1**
8. **Add test-time augmentation (TTA)** if you want an advanced improvement
9. **Clean the labels** for difficult categories like MiscellaneousTrash
10. **Merge / redefine ambiguous labels only if justified by the confusion matrix**

Important project insight

If your biggest errors come from:

- cluttered scenes,
- multiple objects,
- object location confusion,

then that is a sign classification may be hitting its limit and **YOLO/object detect**:

Cell 56 — Markdown

23. Optional: advanced augmentation ideas

You do **not** need these on day one, but they are useful later.

- **MixUp**: blends two training images and labels
- **CutMix**: pastes part of one image into another
- **Random Erasing**: forces robustness to missing regions

These can improve generalization, but they also make the notebook more complex. I recommend getting a strong clean baseline first, then testing these only after you

Cell 57 — Markdown

24. How this connects to YOLO later

Your uploaded YOLO notebook shows the beginning of the next stage, but YOLO solves a

Classification vs. YOLO

- **Classification**: predicts one label for the whole image.
- **YOLO detection**: predicts one or more objects plus bounding boxes.

Why classification first is still the right move

This notebook helps you answer:

- Which classes are hardest?
- Which classes are visually similar?
- Is background clutter hurting performance?
- Is label ambiguity a problem?

Those answers tell you whether YOLO is worth the extra labeling effort.

For the future YOLO step

You will need:

- images,
- bounding-box annotations,
- train/val split in YOLO format,
- object-detection metrics such as mAP.

Cell 58 — Markdown

25. Final recommendations

If your goal is **highest practical accuracy** for the first serious version of this

1. Start with **EfficientNet-B0** as a fast strong baseline.
2. Then try **EfficientNet-B2** for a possible accuracy improvement.
3. Keep the **same stratified split** for every experiment.
4. Record every change in a results table.
5. Use the confusion matrix to decide what to improve next.
6. Only move to YOLO after the classification pipeline is well understood.

That gives you a clean project story for your report and presentation:

- baseline,
- improvements,
- analysis,
- final best model,
- future object detection plan.

Two important things I noticed while reproducing it exactly:

1. Cell 38 has a small string formatting issue

It currently says:

```
print("
" + "=" * 80)
```

It should be:

```
print("\n" + "=" * 80)
```

2. Cell 51 has a small multiline f-string issue

It currently says:

```
axes[i].set_title(f"Pred: {pred_name}
True: {true_name}", color=color)
```

It should be:


```
axes[i].set_title(f"Pred: {pred_name}\nTrue: {true_name}", color=color)
```

So the notebook content above is the notebook **as sent**, but those two lines should be corrected before running.

For convenience, I also saved the full cell-by-cell text dump here:

</>
realwaste_cnn_cells.txt

CELL 1 [MARKDOWN]

RealWaste CNN Classification Notebook

This notebook is designed for your ****waste classification final project**** and is meant to be both:

1. a ****strong training notebook**** focused on ****high classification accuracy****, and

2. a ****learning notebook**** that explains ***why*** each step exists so you can use it in your report and presentation.

I can also produce a **cleaned "professor-ready" version in chat** next, with those two fixes applied and with numbered sections formatted for printing.



show me where data augmentation was used and data preparation was used. we did horizontal flips, rotations, and crops, correct? with this code

```
# Change image size here if you want to experiment.
# 224 is standard and efficient. 260 or 288 can
sometimes help slightly.
IMAGE_SIZE = 224
```

```
train_transform = transforms.Compose([
```

[Show more](#) ▾

Thought for a couple of seconds >

Yes — your reading is correct.

Where data augmentation was used

It was used in the **training transform** here:

```
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.75, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.10, hue=0.02),
    transforms.RandomPerspective(distortion_scale=0.15, p=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

What augmentations are happening there

You specifically asked whether you did horizontal flips, rotations, and crops. The answer is **yes**:

- `RandomResizedCrop(...)`

This does a **random crop** and then resizes it to `IMAGE_SIZE`. So yes, cropping was used.

- `RandomHorizontalFlip(p=0.5)`

This does a **left-right flip** with 50% probability. So yes, horizontal flipping was used.

- `RandomRotation(15)`

This randomly rotates the image up to about **±15 degrees**. So yes, rotation was used.

But there were also **two more augmentation steps** beyond those three:

- `ColorJitter(...)`

Slightly changes brightness, contrast, saturation, and hue.

- `RandomPerspective(...)`

Adds a small perspective distortion.

Then these two are not really “augmentation” in the same sense, but they are still part of preprocessing:

- `ToTensor()`
Converts the image into a PyTorch tensor.
- `Normalize(...)`
Standardizes pixel values using ImageNet mean/std, which is important because the pretrained models expect this.

Where augmentation was *not* used

It was **not** used for validation or test images:

```
val_test_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

That is important because:

- training data gets random variation to improve generalization,
- validation/test data should stay stable and consistent for fair evaluation.

So for validation/test, you only did:

- resize,
- tensor conversion,
- normalization.

No random augmentation there.

Where data preparation was used

You are also correct about the **stratified split**.

This code:

```
train_df, temp_df = train_test_split(
    _df,
    test_size=0.20,
```

```

        stratify=_df['label'],
        random_state=SEED,
    )

    val_df, test_df = train_test_split(
        temp_df,
        test_size=0.50,
        stratify=temp_df['label'],
        random_state=SEED,
    )

```

means:

- first split into:
 - **80% train**
 - **20% temporary**
- then split that 20% temporary into:
 - **10% validation**
 - **10% test**

Because `stratify=...` was used both times, class proportions are preserved as much as possible across train, validation, and test.

That is definitely part of **data preparation**.

What else in data preparation was done?

There was actually quite a bit more data preparation than just augmentation and splitting.

1. Building a dataframe of file paths and labels

This part is data preparation:

```

records = []
for class_dir in sorted([p for p in DATA_DIR.iterdir() if p.is_dir()]):
    for img_path in sorted(class_dir.rglob("*")):
        if img_path.suffix.lower() in IMG_EXTS:
            records.append({"filepath": str(img_path), "label": class_dir.name})

_df = pd.DataFrame(records)

```

What it does:

- scans the dataset folders,
- finds image files,
- records each image path,
- assigns its class label from the folder name,
- stores everything in a dataframe.

Why this matters:

- gives you structured metadata,
- makes stratified splitting possible,
- makes experiment tracking easier.

2. Filtering for valid image file types

This is also data preparation:

```
IMG_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
```

Then only files with those extensions are included.

Why this matters:

- avoids accidentally reading non-image files,
- keeps the dataset clean.

3. Inspecting class balance

This section is part of data understanding and preparation:

```
class_counts = _df['label'].value_counts().sort_index()
```

and then plotting it.

Why this matters:

- shows imbalance across classes,

- helps justify weighted sampling and class-weighted loss.

So even though this is not transforming the images, it is still part of **preparing the dataset for training**.

4. Saving the train/val/test split

This was another data preparation step:

```
train_df.to_csv(SPLIT_DIR / "train_split.csv", index=False)
val_df.to_csv(SPLIT_DIR / "val_split.csv", index=False)
test_df.to_csv(SPLIT_DIR / "test_split.csv", index=False)
```

Why this matters:

- keeps your experiments reproducible,
- ensures future models use the exact same split,
- makes comparisons fair.

For a class project, this is a very good practice.

5. Creating class-to-index mappings

This is data preparation too:

```
class_names = sorted(_df['label'].unique().tolist())
class_to_idx = {name: i for i, name in enumerate(class_names)}
idx_to_class = {i: name for name, i in class_to_idx.items()}
```

What it does:

- converts class names like "Plastic" into integer labels like 6 ,
- creates the reverse mapping too.

Why this matters:

- neural networks train on numeric labels, not strings,
 - needed for loss calculation and prediction decoding.
-

6. Image resizing

This is also part of data preparation / preprocessing:

```
IMAGE_SIZE = 224
```

and then:

```
transforms.Resize((IMAGE_SIZE, IMAGE_SIZE))
```

or the crop-resize in training.

Why this matters:

- all images must be a consistent size,
 - pretrained CNNs expect fixed input dimensions.
-

7. Converting images to tensors

This line is preprocessing:

```
transforms.ToTensor()
```

Why this matters:

- converts PIL images into PyTorch tensors,
 - required before feeding images into the model.
-

8. Normalizing images

This is very important preprocessing:

```
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

Why this matters:

- scales the image channels to the distribution expected by ImageNet-pretrained models,
- helps training stability and performance.

This is one of the most important preparation steps for transfer learning.

9. Creating a custom dataset class

This is part of the input pipeline:

```
class WasteDataset(Dataset):  
    def __init__(self, df, class_to_idx, transform=None):  
        ...
```

and especially:

```
image = Image.open(row['filepath']).convert('RGB')  
label = self.class_to_idx[row['label']]
```

What it does:

- reads images from disk,
- converts them to RGB,
- converts string labels to integers,
- applies transforms.

This is definitely part of data preparation.

Important detail:

```
.convert('RGB')
```

This ensures every image has 3 channels, which is important because pretrained CNNs expect RGB input.

10. Weighted sampling for imbalanced classes

This is another data-preparation/training-input step:

```
train_class_counts = train_df['label'].value_counts().sort_index()
train_sample_weights = train_df['label'].map(lambda x: 1.0 / train_class_counts[x]).\
train_sample_weights = torch.DoubleTensor(train_sample_weights)

weighted_sampler = WeightedRandomSampler(
    weights=train_sample_weights,
    num_samples=len(train_sample_weights),
    replacement=True,
)
```

Why this matters:

- minority-class examples are sampled more often,
- helps reduce the effect of class imbalance.

This is not augmentation, but it is definitely part of how the training data was prepared and fed to the model.

11. Building dataloaders

This is part of data pipeline preparation:

```
train_loader = DataLoader(...)
val_loader = DataLoader(...)
test_loader = DataLoader(...)
```

Why this matters:

- packages the data into batches,
- handles shuffling/sampling,
- prepares efficient loading for training.

Clean summary

So in your notebook, **data preparation** included all of these:

Data organization

- scanning folders and collecting file paths
- assigning labels from folder names
- filtering valid image types
- creating a dataframe

Dataset splitting

- stratified train/validation/test split
- saving split CSV files for reproducibility

Label preparation

- mapping class names to integer indices

Image preprocessing

- converting images to RGB
- resizing / cropped resizing
- converting to tensor
- normalization

Data augmentation for training

- random resized crop
- random horizontal flip
- random rotation
- color jitter
- random perspective distortion

Imbalance handling / training input prep

- class count inspection
- weighted random sampler
- class weights for the loss

The direct answer to your question

Yes, with that code you **definitely** used:

- horizontal flips

- rotations
- crops

But you also used:

- color jitter
- perspective distortion
- normalization
- tensor conversion
- resizing for val/test

And beyond augmentation, the notebook also used data preparation through:

- dataframe creation from filepaths/labels
- stratified splitting
- class label encoding
- RGB conversion
- weighted sampling
- dataloader construction

A strong report sentence would be:

Data preparation included organizing image paths and labels into a dataframe, applying a stratified train/validation/test split, encoding labels numerically, converting images to RGB, resizing and normalizing inputs, and addressing class imbalance through weighted sampling. Data augmentation was applied only to the training set using random resized crops, horizontal flips, rotations, color jitter, and perspective distortion.

I can also turn this into a **report-ready paragraph** and a **bullet list for your presentation slide**.



where in this notebook is the tuning done?

Thought for a few seconds >

The tuning in this notebook is mostly set up in the **configuration and experiment comparison sections**. It is not a giant automatic hyperparameter search; it is **manual**,

controlled tuning so you can test changes one at a time and explain them in your report.

Where the tuning is done

1. Transform tuning

This is one place tuning happens:

```
IMAGE_SIZE = 224

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.75, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.10, hue=0.02),
    transforms.RandomPerspective(distortion_scale=0.15, p=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

This is tuning because you can change:

- IMAGE_SIZE
- crop scale
- flip probability
- rotation amount
- color jitter strength
- perspective distortion amount

So augmentation strength is one tuning area.

2. Training hyperparameter tuning in TrainConfig

This is the main tuning block:

```
@dataclass
class TrainConfig:
    model_name: str = 'efficientnet_b2'
```

```
epochs_head: int = 5
epochs_finetune: int = 12
lr_head: float = 1e-3
lr_finetune: float = 2e-4
weight_decay: float = 1e-4
label_smoothing: float = 0.05
patience: int = 5
use_class_weights: bool = True
```

This is where the notebook exposes the main settings you tune:

- **model backbone:** `model_name`
- **number of epochs:** `epochs_head` , `epochs_finetune`
- **learning rates:** `lr_head` , `lr_finetune`
- **regularization:** `weight_decay`
- **label smoothing:** `label_smoothing`
- **early stopping patience:** `patience`
- **class weighting on/off:** `use_class_weights`

This is probably the most important answer to your question.

3. Model selection tuning

This section tunes **which CNN architecture works best:**

```
MODEL_CANDIDATES = [
    'mobilenet_v3_large',
    'resnet50',
    'efficientnet_b0',
    'efficientnet_b2',
]
```

Then here:

```
for model_name in MODEL_CANDIDATES:
    ...
    cfg = copy.deepcopy(BASE_CONFIG)
    cfg.model_name = model_name
```

</> Python



Run

That is tuning by **comparing different pretrained models** under the same pipeline.

So yes, architecture comparison is one of the main forms of tuning in the notebook.

4. Base experiment settings

This section is another tuning control point:

```
BASE_CONFIG = TrainConfig(
    epochs_head=4,
    epochs_finetune=8,
    lr_head=1e-3,
    lr_finetune=2e-4,
    weight_decay=1e-4,
    label_smoothing=0.05,
    patience=4,
    use_class_weights=True,
)
```

This is where you would manually change values and rerun experiments.

For example, this is exactly where you would tune:

- increase epochs_finetune
- reduce lr_finetune
- test label_smoothing=0.0
- test use_class_weights=False

5. Single-model focused tuning

This commented block is another place meant for tuning:

```
# chosen_config = TrainConfig(
#     model_name='efficientnet_b2',
#     epochs_head=5,
#     epochs_finetune=12,
#     lr_head=1e-3,
#     lr_finetune=2e-4,
#     weight_decay=1e-4,
#     label_smoothing=0.05,
#     patience=5,
#     use_class_weights=True,
# )
# best_model, best_history = run_training(chosen_config, train_loader, val_loader, de
```

This is where you tune one specific model more carefully after the comparison stage.

So the workflow is:

- first compare several backbones,
- then pick one,
- then tune its settings more deeply.

6. Fine-tuning strategy itself

This part is also tuning-related:

```
model = build_model(config.model_name, NUM_CLASSES, freeze_backbone=True)
```

and later inside `run_training`:

```
# Phase 1: head only
...
# Phase 2: fine-tune all layers
for param in model.parameters():
    param.requires_grad = True
```

This means the notebook is also tuning the **training strategy**:

- first train only the classifier head
- then unfreeze and fine-tune all layers

That is not just implementation detail — it is a tuning choice meant to improve accuracy.

What is *not* in the notebook yet

The notebook does **not** do automatic tuning like:

- grid search
- random search
- Optuna
- Bayesian optimization

So the tuning is currently:

- **manual**
- **structured**
- **experiment-driven**

That is actually a good choice for a class project because it is easier to explain.

Best answer in one sentence

The tuning is mainly done through the `TrainConfig` settings, the `BASE_CONFIG` experiment block, the `MODEL_CANDIDATES` backbone comparison loop, and the transform definitions where augmentation and image size can be adjusted.

If you want to point to exact notebook sections

You can say tuning was done in these parts of the notebook:

- **Cell 22**: augmentation and image size tuning
 - **Cell 34**: tunable training hyperparameters in `TrainConfig`
 - **Cell 37**: experiment baseline settings in `BASE_CONFIG`
 - **Cell 36 / 38**: model comparison across CNN backbones
 - **Cell 40**: focused tuning for one chosen final model
 - **Cell 55**: suggested next tuning experiments
-

Report-ready wording

You could write:

Hyperparameter tuning in the notebook was performed manually through controlled experiments. The main tuning points included image size and augmentation settings, model backbone selection, head and fine-tuning epochs, learning rates, weight decay, label smoothing, early stopping patience, and whether to use class weights. Models were compared under the same stratified split so that performance differences could be attributed to the tuning changes rather than differences in the dataset split.

I can also mark up the exact cells as "tuning," "data prep," and "augmentation" so you have a clean professor-facing breakdown.

